

BASIC PANDAS COMMANDS

Think of **pandas** = **Excel inside Python**

(DataFrame = Sheet, Series = Column)

1. Importing Pandas

import pandas as pd

Why pd?

It is an **alias** (short name).

Instead of writing:

pandas.DataFrame()

we write:

pd.DataFrame()

Industry standard - always use pd

2. Series (Single Column Data Structure)

A **Series** is like **one column in Excel**.

```
s = pd.Series([10,20,30,40])
```

```
print(s)
```

Output:

```
0 10
```

```
1 20
```

```
2 30
```

```
3 40
```

Important Concepts

Part	Meaning
Left numbers	Index
Right numbers	Values

Pandas automatically assigns index starting from 0.

You can also give custom index:

```
s = pd.Series([10,20,30], index=["A","B","C"])
```

Now:

```
A 10
```

```
B 20
```

```
C 30
```

Exam Question: What is Series?

One-dimensional labeled array.

3. DataFrame (Most Important Concept)

A **DataFrame** is a **table** (rows + columns).

```
data = {
```

```
    "Name": ["Aman","Riya","Karan"],
```

```
    "Marks": [85,90,78],
```

```
    "City": ["Delhi","Mumbai","Punjab"]
```

```
}
```

```
df = pd.DataFrame(data)
```

Output:

Index	Name	Marks	City
0	Aman	85	Delhi
1	Riya	90	Mumbai
2	Karan	78	Punjab

Internal Structure

DataFrame = collection of Series

DataFrame

```
|— Series(Name)
|— Series(Marks)
|— Series(City)
```

Exam Definition:

Two-dimensional labeled data structure consisting of rows and columns.

4. Reading Files

Real life data comes from files - not manual typing.

CSV (most common)

```
df = pd.read_csv("data.csv")
```

Excel

```
df = pd.read_excel("data.xlsx")
```

JSON

```
df = pd.read_json("data.json")
```

Very Important Concept

Pandas automatically:

- detects column names
 - assigns data types
 - creates index
-

5. Viewing Data (Understanding Dataset)

head() and tail()

```
df.head()
```

Shows first 5 rows.

Why?

Datasets may contain millions of rows → we only preview.

```
df.tail()
```

Shows last rows.

shape

```
df.shape
```

Output:

```
(3, 3)
```

Meaning:

(rows, columns)

columns

df.columns

Shows column names.

dtypes

df.dtypes

Tells data type:

Type	Meaning
int64	numbers
float64	decimal
object	string
bool	true/false

info()

df.info()

Most useful debugging command.

Shows:

- non-null count
 - datatype
 - memory usage
-

describe()

df.describe()

Gives statistics:

Value	Meaning
mean	average
std	spread
min	smallest
max	largest

Used in ML preprocessing

6. Selecting Data (Core Skill)

Selecting Columns

df["Name"]

Returns Series.

df[["Name", "Marks"]]

Returns DataFrame.

Double bracket = table

Single bracket = column

Selecting Rows

iloc → position based (like array index)

df.iloc[0]

First row

```
df.iloc[0,1]
```

Row 0, Column 1

loc → label based (uses names)

```
df.loc[0,"Name"]
```

Exam Difference

iloc	loc
index position	label name
numbers only	names allowed

Conditional Selection (MOST IMPORTANT)

```
df[df["Marks"] > 80]
```

Meaning:

Filter rows where marks > 80

Internally pandas creates boolean mask:

```
[True, True, False]
```

Then keeps only True rows.

This is called **Boolean Indexing**

7. Adding & Modifying Data

Add Column

```
df["Grade"] = ["A","A","B"]
```

Pandas automatically aligns rows.

Create Column Using Logic

```
df["Result"] = df["Marks"] > 80
```

Creates:

Marks	Result
85	True
90	True
78	False

Important in ML feature engineering

Update Values

```
df.loc[0,"Marks"] = 95
```

Always use loc for safe editing.

8. Delete Data

Delete Column

```
df.drop("City", axis=1, inplace=True)
```

Why axis=1?

axis	meaning
0	row

1	column
---	--------

inplace=True

Apply change permanently.

Delete Row

`df.drop(0, axis=0, inplace=True)`

9. Sorting

`df.sort_values("Marks")`

Ascending by default.

`df.sort_values("Marks", ascending=False)`

Descending.

Used heavily in data analysis.

10. Missing Values Handling

Real datasets always contain missing values.

Detect Missing

`df.isnull()`

Returns True/False table.

`df.isnull().sum()`

Counts missing values.

Remove Missing

`df.dropna()`

Deletes rows with empty values.

Fill Missing

`df.fillna(0)`

Replace empty with 0.

`df.fillna(method="ffill")`

Forward fill:

copies previous value downward.

Important preprocessing step in Machine Learning

11. Grouping & Aggregation (SQL-like feature)

`df.groupby("City")["Marks"].mean()`

Meaning:

Find average marks city-wise

Internally pandas:

1. splits data into groups
2. applies function
3. combines result

This is called **Split-Apply-Combine**

Examples:

```
df.groupby("City")["Marks"].sum()
df.groupby("City").count()
```

12. Saving Files

After processing data:

```
df.to_csv("output.csv", index=False)
```

```
df.to_excel("output.xlsx", index=False)
```

index=False prevents writing extra numbering column.

Final Conceptual Summary

Concept	Real-world meaning
Series	single column
DataFrame	Excel sheet
loc	search by name
iloc	search by position
groupby	category analysis
fillna	repair missing data
describe	statistics
boolean indexing	filtering
drop	delete
sort_values	ranking
